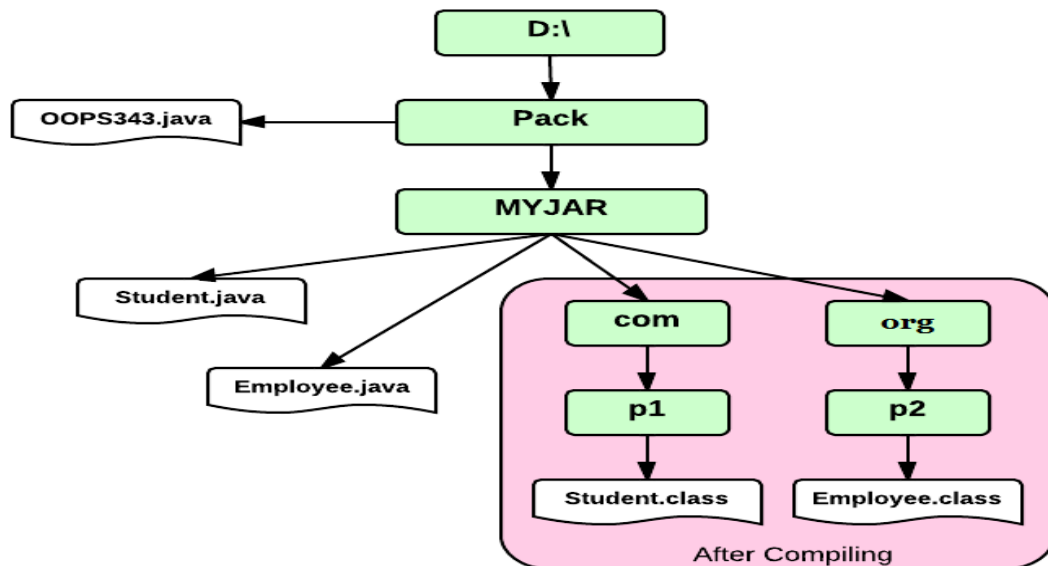


JAR (Java Archive)

- ❖ Package is the collection of classes where JAR is the collection of package.
- ❖ JAR file format is platform-independent.
- ❖ You can compress and bundle multiple files associated with a java application into a single file called as **JAR file**.
- ❖ It is based on the ZIP algorithm.



Syntax:

```

jar -cf <Name of JAR File><pack1><pack2> ...
jar -cvf <Name of JAR File><pack1><pack2> ...
  
```

Programs:

<u>Student.java</u> <pre> package com.p1; public class Student{ public int sid; public String sname; public void show(){ System.out.println("Student->show()"); System.out.println("sid : "+sid); System.out.println("sname : "+sname); } } </pre>	<u>Employee.java</u> <pre> package com.p1; public class Employee{ public int eid; public String ename; public void show(){ System.out.println("Employee->show()"); System.out.println("eid : "+eid); System.out.println("ename : "+ename); } } </pre>
<u>OOPS343.java</u> <pre> package com.p2; import com.p1.*; import org.p2.*; public class OOPS343{ public static void main(String args[]){ Student st=new Student(); </pre>	<pre> st.sid=99; st.sname="SAI"; st.show(); Employee emp=new Employee(); emp.eid=99; emp.ename="RAM"; emp.show(); }} </pre>

Step for creating jar file and executing.

1. Compile java program with following command

```
javac -d . Student.java
javac -d . Empolyee.java
javac -d . OOPS343.java
Or
javac -d . *.java
```

2. Creating jar file with following command

```
jar -cf sai.jar com org
Or
jar -cvf sai.jar com org
```

3. Delete the file which is created on compile time. But do not delete sai.jar file.
4. You need to set the CLASSPATH for the jar file to access the package from JAR file.

```
SET CLASSPATH=%CLASSPATH%; D:\pack\sai.jar;
```

5. Now the classes available in JAR file will be accessed by compiler or JVM

```
javac -d . OOPS343.java          =>Compile Successfully
java com.p3.OOPS343.java        =>Runs Successfully
```

Output: -

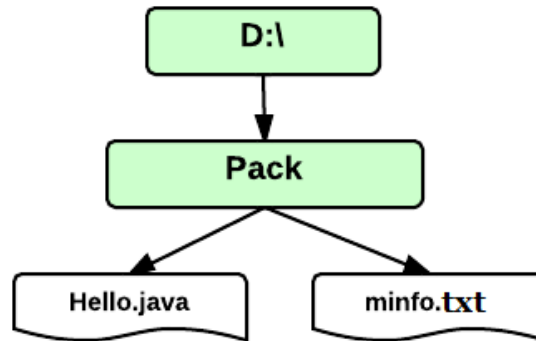
<pre>Student->show() sid : 99 sname : SAI Employee->show() eid : 88 ename : RAM</pre>	 This type of jar file will created in define directory.
---	--

Creating the Executable JAR file:

- ❖ When the jar file is executable then you need to specify the Main class information.
- ❖ Main class information will be stored in then MANIFEST .MF file internally.
- ❖ When you try to execute JAR then JVM will search main class from the MANIFEST .MF file.
- ❖ To specify the manifest information you need to create a text file with Main Class information.
- ❖ When any problem occurred in manifest file while creating executable jar file then “java.io.IOException: invalid header” error will be given at runtime.
- ❖ Make sure that you have pressed Enter after class name in the manifest file otherwise while executing you may get “no manifest attribute” error.

Example:

Main-Class: com.p1.OOPS344 ↴
 ↑
 1 Space Press Enter

**OOPS344.java**

```
package com.p2;
import javax.swing.JOptionPane;
public class OOPS344{
    public static void main(String args[]){
        String val1=JOptionPane.showInputDialog("Enter First Int Value");
        String val2=JOptionPane.showInputDialog("Enter 2nd Int Value");
        int a=Integer.parseInt(val1);
        int b=Integer.parseInt(val2);
        int c=a+b;
        String msg="Sum of "+a+" and "+b+" is "+c;
        JOptionPane.showMessageDialog(null,msg);
    }
}
```

minfo.txt

Main-Class: com.p1.OOPS344

Compile the Java program

```
javac -d . OOPS344.java
```

Create the JAR file:

```
jar -cvmf minfo.txt jlc.jar com
```

Execute JAR file:

```
cmd > java -jar Exjar.jar
```

or

Double click on executable jar file.

Extract JAR file:

```
jar -xvf Exjar.jar
```

Output: